

CSC 406 - Banking Systems

BankApp Programmer's Manual

Java Edition • Version 1.2 • Draft

Branch: prototypeBranch

Audience: Software developers and engineers contributing to BankApp.

Language: Java - JDK 11+

Status: Draft - v1.2 updated per project lead feedback and prototypeBranch code.

1. Introduction

1.1 Purpose

This manual is the technical reference for developers working on BankApp, a Java-based banking platform. It documents the system's design decisions, class hierarchies, interaction patterns, and behavioural workflows as expressed in the four UML artifacts produced for CSC 406. This version has been updated to reflect the implemented code on prototypeBranch.

1.2 Audience & Prerequisites

Written for software engineers maintaining or extending BankApp. Readers should have:

- Proficiency in Java (JDK 11+) and object-oriented design principles.
- Working knowledge of UML notation: class, sequence, activity, and use case diagrams.
- Basic understanding of relational databases and REST API conventions.

1.3 Application Overview

BankApp manages four primary domains:

- Accounts: CheckingsAccount (abstract) with concrete subtypes TMBAccount and GDAccount (Gold Diamond), plus SavingsAccount and CDAccount - all extending an abstract Account base.
- Loans: TermBasedLoan (abstract) extended by MortgageLoan and ShortTermLoan; CreditCard extends Loan directly.
- Transactions: Deposits, withdrawals, transfers, overdraft handling, CC charges, and loan payments.
- Data Persistence: CSV-based flat-file storage via ArrayListManager and a PrintUtil helper. A JavaFX UI entry point (MainApp) is also present.

1.4 Core Objectives

- Correctness and consistency of financial data across all transaction types.
- Extensibility through abstraction - new account and loan types extend existing base classes.
- Role-based security - Customer, Teller, and Bank Manager hold distinct operation privileges (UML design; not yet implemented in prototype).
- Intelligent fee management - including overdraft fee logic in CheckingsAccount.requestFeeWaiver().

1.5 Glossary

Term	Definition
Account	Abstract base class with deposit(), withdraw(), getBalance(). accountNumber generated via UUID.
CheckingsAccount	Abstract checking base. Handles overdraft protection, stop-payment stub, transfer(), makeMonthlyTransfer(), and requestFeeWaiver().

TMBAccount	Concrete checking with a \$0.75 per-transaction fee and \$1.25 monthly transfer fee.
GDAccount	Gold Diamond checking with \$5,000 minimum balance, linked interest savings account, and flexible daily interest rate.
SavingsAccount	Concrete savings with configurable interest rate and overdraft backup flag.
CDAccount	Fixed-rate CD with maturity date, early-withdrawal penalty, and rollover().
Loan	Abstract loan base. Manages principal, interest rate, currentPaymentDue, makePayment(), closeAccount().
TermBasedLoan	Abstract term-loan subclass. Calculates amortised monthly payment, supports loanPaymentCycle(), flagIfMissed(), applyLateFee().
MortgageLoan	Term-based loan restricted to 15 or 30-year terms.
ShortTermLoan	Term-based loan with a fixed 5-year term.
CreditCard	Revolving credit with credit limit, transaction list, finance charge calculation, and bill generation.
Customer	User class with full address fields, account list, CSV serialisation, and Comparable sort by last name.
ArrayListManager	Generic sorted collection utility used to maintain the customer list.
PrintUtil	Utility class that writes a customer and account summary to disk.
MainApp	JavaFX entry point. Launches the SystemController FXML UI.

2. System Architecture

BankApp is organised around three concerns: the domain model (Account/Loan hierarchies), the data layer (CSV persistence + ArrayListManager), and the emerging UI layer (JavaFX via MainApp). The four UML diagrams below are the authoritative architectural reference for intended system behaviour; the subsections note where the implemented code aligns or deviates.

2.1 Class Diagram - Domain Model

Account Hierarchy

Account (abstract) defines deposit(), withdraw(), and getBalance(). withdraw() now has a full implementation - it rejects non-positive amounts and correctly subtracts from the private balance (a change from the earlier stub). The accountNumber field is auto-generated using UUID.randomUUID().

Three concrete lineages branch from Account:

- SavingsAccount - interest rate and compound frequency, overdraftBackup flag, applyDailyInterest(), toggleOverdraftBackup(). A default no-arg constructor with randomised values has been added for testing convenience.
- CDAccount - fixedRate, maturityDate, earlyPenalty. rollover() prints a notice. earlyWithdraw() is a stub. withdraw() is overridden to refuse withdrawals (CDAccount funds must exit via earlyWithdraw()).
- CheckingsAccount (abstract) - overdraftProtAccount (SavingsAccount reference), minimumBalance, handleOverdraft(), transfer(), makeMonthlyTransfer(), requestFeeWaiver(double checkAmt), stopPayment() stub, setOverdraftProtAccount() / getOverdraftProtAccount().
- TMBAccount (extends CheckingsAccount) - tsFee = \$0.75 applied per transfer; monthlyTransferFee = \$1.25 applied per monthly transfer. overdraftIdentifier field present.
- GDAccount (extends CheckingsAccount) - minimumBalance = \$5,000 hard-coded. interestAccount (linked SavingsAccount), dailyRateFlexible flag. applyInterest() derives rate from the interest account balance. chargeBelowMinFee() charges \$0.75 on transfer and \$1.25 on monthly transfer when balance is below minimum.

Loan Hierarchy

Loan (abstract) holds principal, interest_rate, currentPaymentDue, isProblemAccount, LoanStatus, and four LocalDate fields. makePayment() is overloaded - one variant uses the current date, the other accepts a LocalDate parameter for testing. closeAccount() returns the total amount still owed. getBalance() returns currentPaymentDue.

TermBasedLoan (abstract, extends Loan) adds term, monthlyPayment, termMonthsLeft, lateFee (\$75.00), and a dueDateInterval. calculateLoanRepayment() computes the standard amortisation formula. loanPaymentCycle() sets the current monthly due and advances the repayment calendar. makePayment() recalculates the schedule when the customer overpays. flagIfMissed() and applyLateFee() both accept an optional LocalDate for test/demo use. MortgageLoan and ShortTermLoan are thin subclasses of TermBasedLoan - MortgageLoan enforces 15 or 30-year terms; ShortTermLoan fixes the term at 5 years.

CreditCard (extends Loan) adds creditLimit, an ArrayList<Transaction> ledger, averageDue, and financeCharge. Key methods: authorizeCharge(), makeTransaction() (overloaded), makePayment() (overloaded), generateBill(), checkBillStatus(), applyInterest(), and getFinanceCharge(). Finance charge is the time-weighted average daily balance for the prior month.

Supporting Classes

- Customer (package User_Classes) - implements Comparable, sorted by lastName. Fields: customerId, address, city, state, zipcode, firstName, lastName, accountList (List<Account>). LineForCSV() serialises all fields as a comma-delimited row. Default constructor generates random test data.
- AppState (package Utils) - singleton holding global data structures shared across all JavaFX controllers. Stores the application-wide ArrayListManager<Customer> list loaded at startup. Controllers access shared state via AppState rather than passing data between scenes.
- Main - developer testing area only. Not part of the production application flow. Used during development to manually exercise CSV I/O and account behaviour.
- MainApp - JavaFX entry point for the production application. Loads SystemController.fxml and opens the Stage.

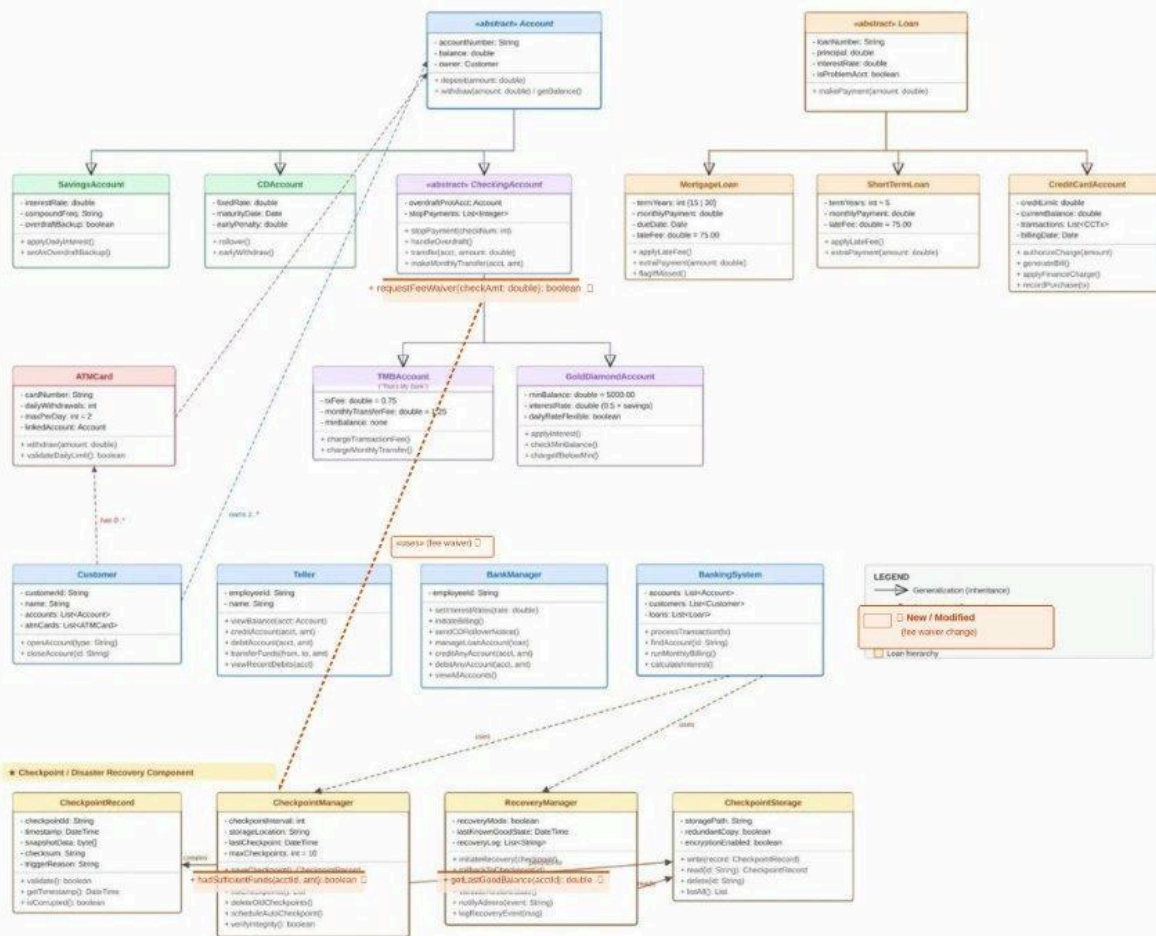


Figure 4 - UML Class Diagram - Full System Hierarchy

2.2 Use Case Diagram - Actor Permissions

Four actors interact within the Banking System boundary: Customer (primary), Teller (extends Customer), Bank Manager (extends Teller), and System (automated). Key relationships:

- Withdraw Money «extends» Overdraft Protection - auto-covers shortfall from savings.
- ATM Withdrawal «extends» Check ATM Daily Withdrawal Limit - enforces per-card cap.
- Make CC Purchase «extends» Waive Overdraft Fee (Checkpoint Verified) - new Scenario H use case (orange). CheckpointManager consulted before any \$25 fee is applied; prior sufficiency waives it entirely.
- Save System Checkpoint «includes» Verify Checkpoint - every save includes checksum verification.
- Restore from Checkpoint «extends» Notify Admins / Log Recovery Event.

CSC 406 // UML Use Case Diagram - Banking System

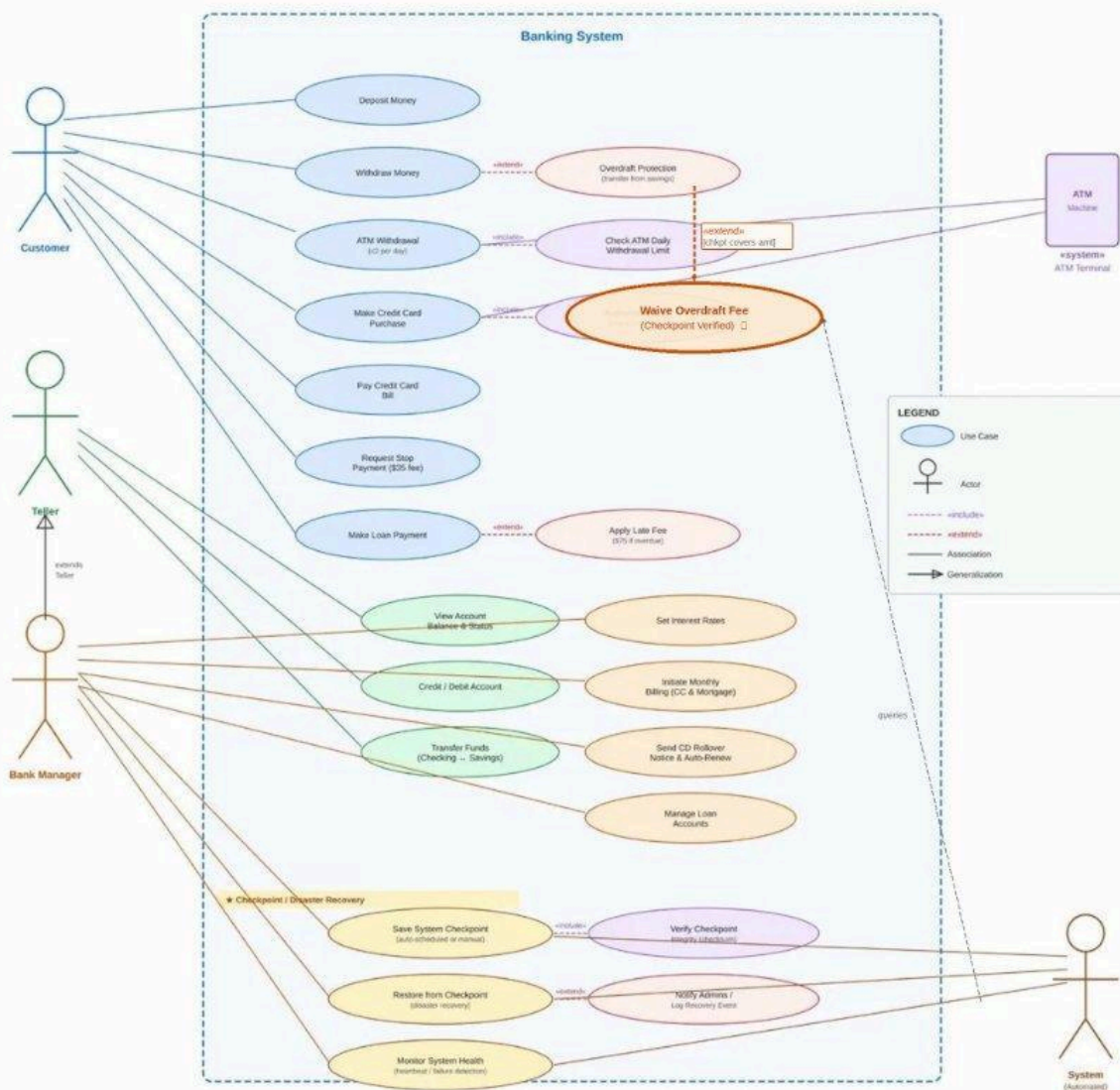


Figure 3 - UML Use Case Diagram - Banking System

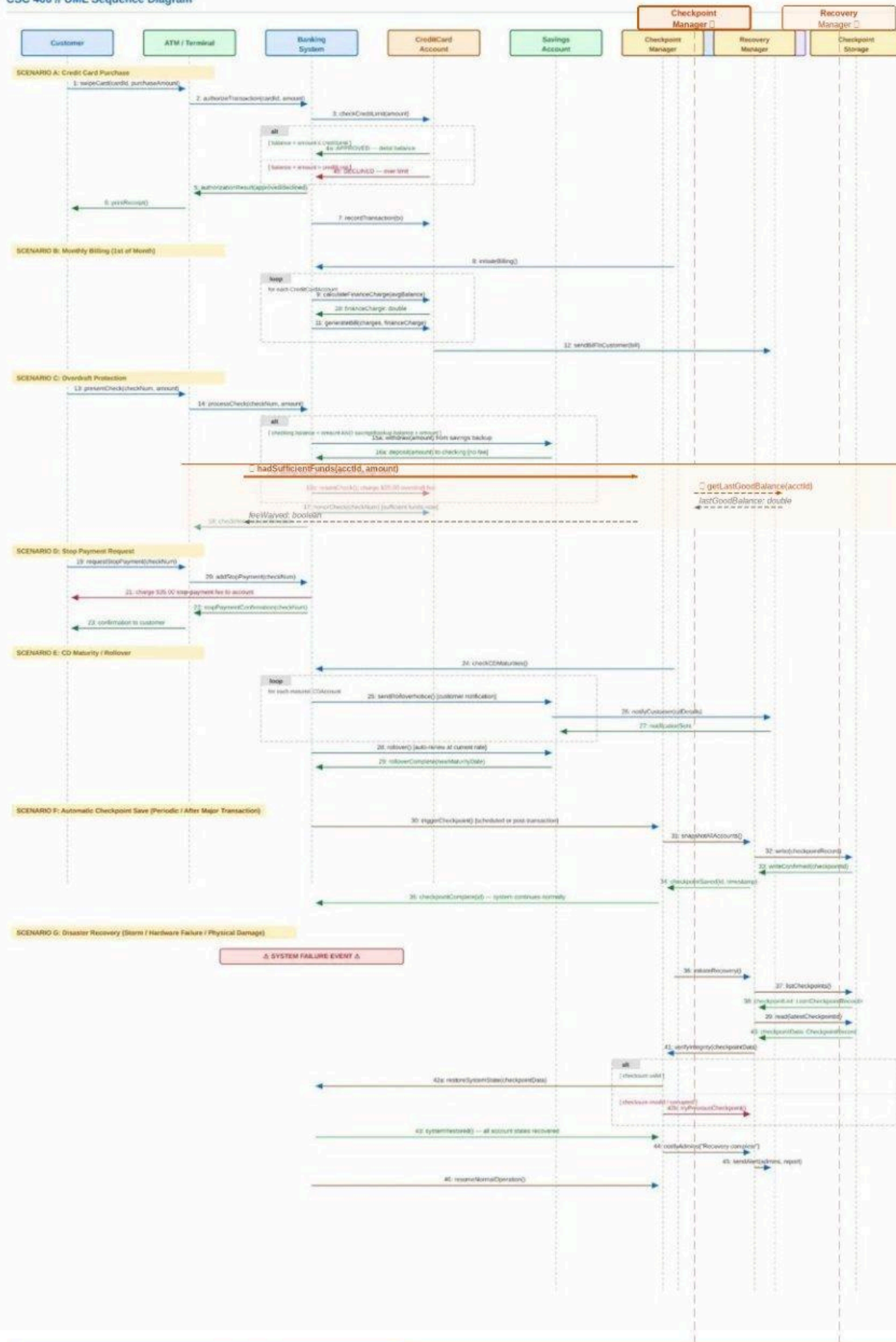
2.3 Sequence Diagram - Runtime Scenarios A–H

Eight scenarios are documented across seven lifelines: Customer, ATM/Terminal, Banking System, CreditCard Account, Savings Account, Checkpoint Manager, Recovery Manager, and Checkpoint Storage.

- A - Credit Card Purchase - swipe → authorizeTransaction → checkCreditLimit (alt: approved/declined) → printReceipt → recordTransaction.
- B - Monthly Billing - initBilling() → loop over each CreditCardAccount → calculateFinanceCharge → generateMonthlyCharges → sendBillToCustomer.
- C - Overdraft Protection - presentCheck → processCheck → alt[savings sufficient]: withdraw/deposit no-fee; else: returnCheck. hasSufficientFunds() overlaid orange.
- D - Stop Payment - requestStopPayment → addStopPayment → charge \$25 → stopPaymentConfirmation → customer confirmation.
- E - CD Maturity / Rollover - checkCDMaturity() → loop: sendRolloverNotice → rollover() → rolloverComplete(newMaturityDate).
- F - Checkpoint Save - triggerCheckpoint → snapshotAllAccounts → writeCheckpointRecord → confirmCheckpoint → checkpointComplete.
- G - Disaster Recovery - SYSTEM FAILURE → lastCheckpoint → readCheckpointData → alt[valid]: restoreSystemState; else: try previous → systemRestored → notifyAdmins.
- H - Overdraft Fee Waiver (New) - hasSufficientFunds(acctId, checkAmt) → getLastGoodBalance(acctId). If lastGoodBalance >= checkAmt: honorCheck(), \$0 FEE WAIVED. Else: returnCheck() + \$25 fee.

Scenario H is detailed in the orange annotation panel at the bottom of Figure 1. The hasSufficientFunds() and getLastGoodBalance() calls are overlaid on Scenario C's lifelines in orange. In the current build, requestFeeWaiver() performs a local balance check (balance - checkAmt >= \$5,000) rather than delegating to CheckpointManager.

CSC 406 // UML Sequence Diagram



SCENARIO H: Overdraft Fee Waiver (New Scenario)

Trigger: Check presented, checking balance insufficient

1. BankingSystem --> CheckpointManager: hadSufficientFunds(acctId, checkAmt)

2. CheckpointManager --> RecoveryManager: getLastGoodBalance(acctId)

3a. [lastGoodBalance >= checkAmt] --> feeWaived = true

BankingSystem withdraws from savings, honorCheck() -- \$0 FEE WAIVED

Figure 1 - UML Sequence Diagram - Scenarios A–H

2.4 Activity Diagram - Transaction Flow & Checkpoint Recovery

Main Transaction Flow

Customer initiates → ATM/Terminal authenticates (fail → Deny Access). On success, four parallel transaction-type paths resolve into Validate Amount & Check Balance. A new decision node - Checkpoint Covers Amt? - calls CheckpointManager.hasSufficientFunds() before any fee is applied (highlighted orange). If confirmed: Honor Check, Fee WAIVED (\$0). Otherwise, return the check + \$25 fee. All paths converge at Generate Confirmation → Log Transaction → 1st-of-Month billing check → Transaction Complete.

Checkpoint / Recovery Flow (Parallel Concern)

Triggered by a scheduled timer, a major transaction, or a manual Bank Manager action.

CheckpointManager: snapshot all accounts → compute checksum → write to CheckpointStorage (redundant copy) → log checkpoint saved. RecoveryManager (on failure): detect → list checkpoints → select latest valid (loop if invalid) → restore system state → restart → notify admins.

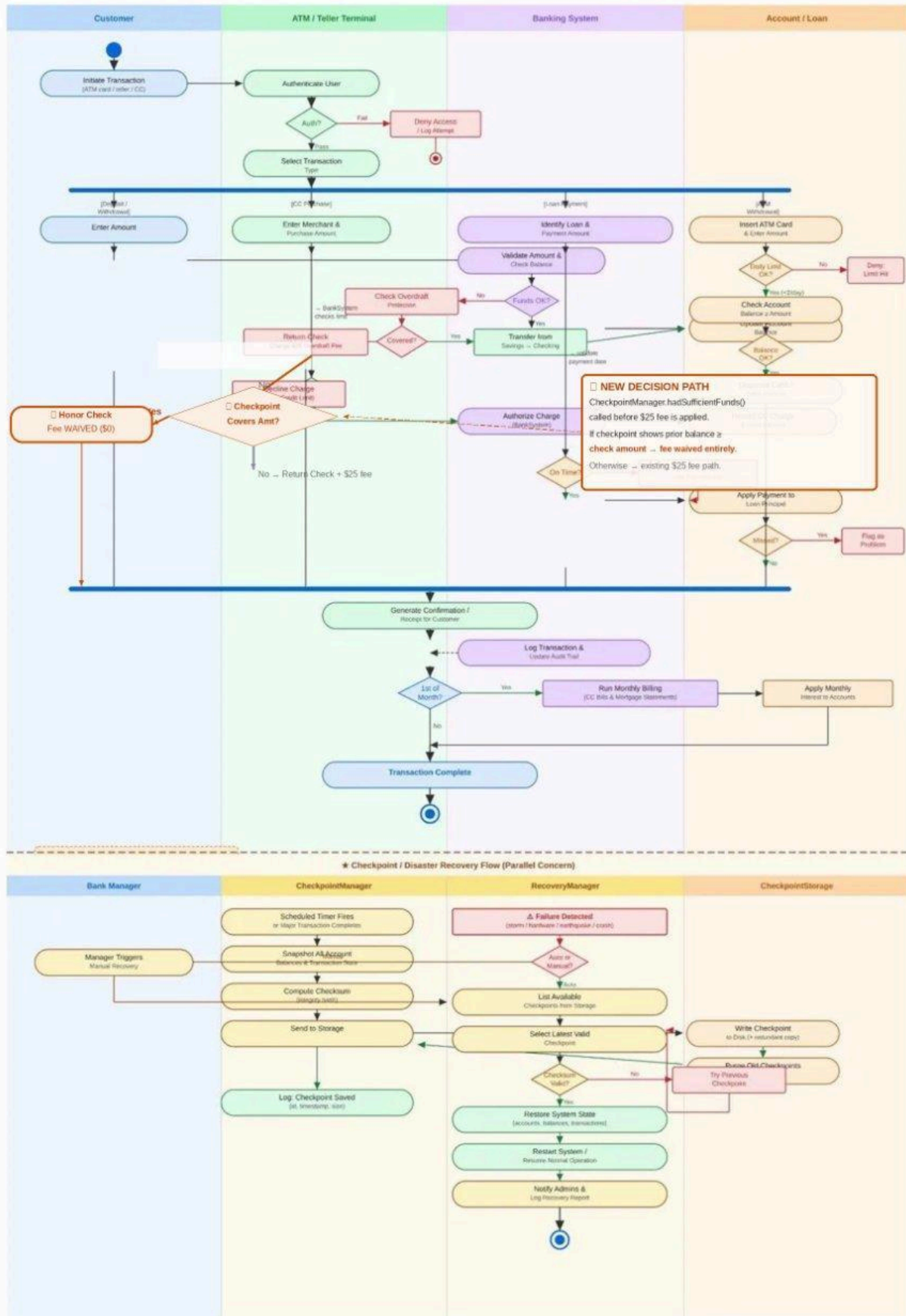


Figure 2 - UML Activity Diagram - Transaction Flow & Recovery

3. Core Modules

This section documents every class implemented on prototypeBranch. Classes are grouped by package. The 'Beyond Prototype Scope' table at Section 3.8 lists UML-specified classes not yet coded.

3.1 Account (abstract) - package Account_Classes

File: src/Account_Classes/Account.java

Field	Type / Description
accountNumber	String - UUID generated at construction time.
balance (private)	double - encapsulated balance; only accessible via getBalance().
Owner	Customer - reference to the owning customer object.

Method	Description
<code>deposit(double amount)</code>	Rejects non-positive amounts. Otherwise adds to balance and prints before/after to stdout.
<code>withdraw(double amount)</code>	Now fully implemented (no longer a stub). Rejects non-positive amounts; subtracts from balance and prints before/after.
<code>getBalance() : double</code>	Returns the private balance.

Change from v1.0: withdraw() was previously a stub. It is now a full implementation with input validation.

3.2 SavingsAccount - package Account_Classes

File: src/Account_Classes/SavingsAccount.java

Field	Type / Description
intrestRate	double - interest rate (note: field name retains original spelling from source). Rate is expressed as a decimal multiplier, e.g. 0.25 = 25%.
compoundFreq	String - compounding frequency label (e.g., "Daily", "31").
overdraftBackup	boolean - flags this account as eligible to cover overdrafts on a linked CheckingsAccount.

Method / Constructor	Description
<code>SavingsAccount()</code>	New default constructor. Generates a random UUID, a random interest rate (0.01–99.99), compoundFreq "31", overdraftBackup true, and a random balance up to \$1,000. Used for quick test instantiation.
<code>SavingsAccount(double intrestRate, String compoundFreq, boolean</code>	Full parameterised constructor. Sets all fields and calls deposit(balance).

<code>overdraftBackup, double balance)</code>	
<code>applyDailyInterest()</code>	Deposits <code>getBalance() * interestRate</code> into the account. Prints before/after balance.
<code>toggleOverdraftBackup()</code>	Flips the <code>overdraftBackup</code> boolean and prints confirmation.

3.3 CDAccount - package Account_Classes

File: `src/Account_Classes/CDAccount.java`

Field	Type / Description
<code>fixedRate</code>	double - fixed interest rate for the CD term.
<code>maturityDate</code>	java.util.Date - the date the CD reaches maturity.
<code>earlyPenalty</code>	double - penalty amount charged on early withdrawal.

Method / Constructor	Description
<code>CDAccount(double balance, double fixedRate, Date maturityDate, double earlyPenalty)</code>	Deposits the opening balance and sets all CD fields.
<code>rollover()</code>	Prints a rollover notice. Full rollover logic is not yet implemented.
<code>earlyWithdraw()</code>	Stub - intended to apply the <code>earlyPenalty</code> and release funds.
<code>withdraw(double amount) [override]</code>	Overrides <code>Account.withdraw()</code> to print "No", preventing direct cash withdrawal. Callers must use <code>earlyWithdraw()</code> instead.

3.4 CheckingsAccount (abstract) - package Account_Classes

File: `src/Account_Classes/CheckingsAccount.java`

Field	Type / Description
<code>overdraftProtAccount</code>	SavingsAccount - linked savings account used to cover overdrafts.
<code>monthlyPayments</code>	List<Integer> - list of scheduled monthly payment amounts.
<code>checkNum</code>	int - current check number counter.
<code>minimumBalance</code>	double - minimum required balance (used by subclasses).

Method / Constructor	Description
<code>CheckingsAccount(SavingsAccount overdraftAccount, double balance)</code>	Generates a UUID, sets the overdraft account, and deposits the opening balance.

<code>stopPayment(int checkNumber)</code>	Stub - intended to add a check number to the stop-payment list.
<code>handleOverdraft()</code>	If balance < 0 and overdraftProtAccount has funds: transfers the deficit from savings. If balance < 0 with no backup: charges a \$25 overdraft fee to this account.
<code>transfer(Account account, double amount)</code>	Withdraws amount, deposits into target account, then calls <code>handleOverdraft()</code> .
<code>makeMonthlyTransfer(Account account, double amount)</code>	Same as <code>transfer()</code> but labelled as a monthly transfer. Calls <code>handleOverdraft()</code> afterwards.
<code>requestFeeWaiver(double checkAmt) : boolean</code>	Returns true if <code>(getBalance() - checkAmt) >= \$5,000</code> . Used to waive the overdraft fee for high-balance accounts. Note: simplified local check; does not yet consult <code>CheckpointManager</code> .
<code>setOverdraftProtAccount(SavingsAccount acct)</code>	Sets the overdraft savings account and calls <code>toggleOverdraftBackup()</code> on it.
<code>getOverdraftProtAccount() : SavingsAccount</code>	Returns the current overdraft protection account.

3.5 TMBAccount - package Account_Classes

File: `src/Account_Classes/TMBAccount.java` | extends `CheckingsAccount`

Field	Type / Description
<code>tsFee</code>	double = 0.75 - fee charged per transfer.
<code>monthlyTransferFee</code>	double = 1.25 - fee charged per monthly transfer.
<code>overdraftIdentifier</code>	int = -1 - reserved identifier field.

Method / Constructor	Description
<code>TMBAccount(SavingsAccount overdraftAccount, double balance)</code>	Delegates to <code>CheckingsAccount</code> constructor.
<code>transfer(Account account, double amount) [override]</code>	Withdraws <code>tsFee</code> (\$0.75) first, then calls <code>super.transfer()</code> .
<code>makeMonthlyTransfer(Account account, double amount) [override]</code>	Withdraws <code>monthlyTransferFee</code> (\$1.25) first, then calls <code>super.makeMonthlyTransfer()</code> .

3.6 GDAccount - package Account_Classes

File: `src/Account_Classes/GDAccount.java` | extends `CheckingsAccount`

Field	Type / Description
<code>interestAccount</code>	<code>SavingsAccount</code> - linked account whose balance drives the daily interest rate.

minimumBalance	double = 5000.0 - minimum balance threshold.
interestRate (private)	double - computed from the interest account balance.
dailyRateFlexible	boolean - indicates whether the daily rate is flexible.

Method / Constructor	Description
<code>GDAccount(SavingsAccount overdraftAccount, double balance, boolean flexibleRate)</code>	Sets dailyRateFlexible and delegates to CheckingsAccount.
<code>setInterestAccount(SavingsAccount account)</code>	Links the savings account whose balance determines interest.
<code>applyInterest()</code>	Sets interestRate = $0.5 \times \text{interestAccount.getBalance()}$. Intended to be called daily.
<code>checkMinBalance()</code>	Stub - minimum balance check logic not yet implemented.
<code>chargeBelowMinFee(double tsFee)</code>	If balance < minimumBalance, withdraws tsFee.
<code>transfer(Account account, double amount) [override]</code>	Calls chargeBelowMinFee(0.75), then super.transfer().
<code>makeMonthlyTransfer(Account account, double amount) [override]</code>	Calls chargeBelowMinFee(1.25), then super.makeMonthlyTransfer().

3.7 Loan (abstract) - package Loan_Classes

File: src/Loan_Classes/Loan.java

Abstract base class for all loan products. Manages the core payment lifecycle - tracking principal, current payment due, and problem account status. Extended by TermBasedLoan and CreditCard.

Fields

Field	Type / Description
principal	double - remaining principal balance.
interest_rate	double - annual interest rate.
currentPaymentDue	double - amount currently owed for this payment cycle.
isProblemAccount	boolean - flagged when payments are overdue. Defaults to false.
LoanStatus	String - status label.
loanRepaymentDate	LocalDate - date of next repayment.
paymentDueDate	LocalDate - due date for the current payment.
dateNotifiedOfPayment	LocalDate - date customer was notified of payment (reserved for future use).
dateSinceLastBalanceChange	LocalDate - tracks when the balance last changed. Used in time-weighted average calculations for finance charges.

Methods

Method	Description
<code>makePayment(double amount) : double</code>	Clears <code>currentPaymentDue</code> first. Remainder is applied to principal. If overpaid, returns the surplus. Updates <code>dateSinceLastBalanceChange</code> to today.
<code>makePayment(double amount, LocalDate paymentDate) : double</code>	Same logic, but records the supplied <code>paymentDate</code> rather than today. Used for testing and presentation.
<code>closeAccount() : double</code>	Returns <code>currentPaymentDue</code> + principal - the total amount still owed to close the account.
<code>getBalance() : double</code>	Returns <code>currentPaymentDue</code> .
<code>getIsProblemAccount() : boolean</code>	Returns the <code>isProblemAccount</code> flag.

3.7a TermBasedLoan (abstract) - package Loan_Classes

File: `src/Loan_Classes/TermBasedLoan.java` | extends `Loan`

Abstract intermediate class for fixed-term amortising loans. Handles the full repayment schedule calculation, monthly payment cycle, late fee application, and problem account flagging.

Fields

Field	Type / Description
<code>term</code>	<code>int</code> - loan term in years.
<code>monthlyPayment</code>	<code>double</code> - calculated amortised monthly payment.
<code>termMonthsLeft</code>	<code>int</code> - months remaining in the loan term.
<code>lateFee</code>	<code>double</code> = 75.00 - fee applied when a payment is overdue.
<code>dueDateInterval</code>	<code>Period</code> - interval between payment due dates.

Methods

Method	Description
<code>calculateLoanRepayment()</code>	Applies the standard amortisation formula ($P \times r(1+r)^n / ((1+r)^n - 1)$) to set <code>monthlyPayment</code> .
<code>loanPaymentCycle()</code>	Sets <code>currentPaymentDue</code> to the calculated <code>monthlyPayment</code> , advances <code>loanRepaymentDate</code> , and reduces principal by the interest portion.
<code>makePayment(double amount) : double [override]</code>	If <code>amount > currentPaymentDue</code> , calls <code>super.makePayment()</code> then recalculates schedule. Otherwise delegates to <code>super</code> .
<code>flagIfMissed()</code>	Marks <code>isProblemAccount = true</code> if <code>today >= loanRepaymentDate</code> and <code>currentPaymentDue > 0</code> .
<code>flagIfMissed(LocalDate currentDate)</code>	Same, using supplied date. For testing/presentation.

<code>disableFlag()</code>	Clears the <code>isProblemAccount</code> flag.
<code>applyLateFee()</code>	Adds <code>lateFee</code> (\$75.00) to <code>currentPaymentDue</code> if today is past <code>paymentDueDate</code> .
<code>applyLateFee(LocalDate currentDate)</code>	Same, using supplied date.
<code>getCurrentPaymentDue() : double</code>	Accessor for <code>currentPaymentDue</code> .

3.7b MortgageLoan - package Loan_Classes

File: `src/Loan_Classes/MortgageLoan.java` | extends `TermBasedLoan`

No additional fields or methods beyond `TermBasedLoan`. Enforces `term` = 15 or 30 years; defaults to 15 if an invalid value is supplied. Provides four constructor overloads varying whether `dueDateInterval` and/or `currentDate` are explicitly provided.

3.7c ShortTermLoan - package Loan_Classes

File: `src/Loan_Classes/ShortTermLoan.java` | extends `TermBasedLoan`

No additional fields or methods. `term` is fixed at 5 years. Four constructor overloads parallel `MortgageLoan`'s pattern. Default `dueDateInterval` is `Period.ofDays(10)`; default repayment start is one month from construction.

3.7d CreditCard - package Loan_Classes

File: `src/Loan_Classes/CreditCard.java` | extends `Loan`

Revolving credit account. Unlike term-based loans, the balance fluctuates with purchases and payments. Finance charges are calculated as a time-weighted average daily balance over the billing period.

Fields

Field	Type / Description
<code>creditLimit</code>	<code>double</code> - maximum allowable balance. New charges are rejected if $(\text{currentPaymentDue} + \text{charge}) > \text{creditLimit}$.
<code>transactions</code>	<code>ArrayList<Transaction></code> - ordered ledger of all charges and payments.
<code>averageDue</code>	<code>double</code> - running time-weighted sum used to compute the finance charge. Updated on every balance change.
<code>financeCharge</code>	<code>double</code> - finance charge accrued for the current billing cycle. Defaults to 0.0. Cleared when paid.

Constructor

Constructor	Description
-------------	-------------

<pre>CreditCard(double currentPaymentDue, double interest_rate, String loanStatus, boolean isProblemAccount, double creditLimit, LocalDate dateCreated)</pre>	Sets all fields. loanRepaymentDate is set to the 10th of the month following dateCreated. averageDue is initialised to currentPaymentDue. dateSinceLastBalanceChange is set to dateCreated.
---	---

Methods

Method	Description
<pre>authorizeCharge(double charge) : boolean</pre>	Returns false and prints 'Insufficient funds!' if (currentPaymentDue + charge) > creditLimit. Returns true otherwise. Called by makeTransaction() before adding any charge.
<pre>generateBill(LocalDate currentDay)</pre>	Must be called on the 1st of the month. If currentPaymentDue > 0, computes the finance charge as the time-weighted average daily balance divided by the number of days in the prior month. If no balance exists, resets averageDue to 0.
<pre>checkBillStatus()</pre>	Sets isProblemAccount = true if today's date is >= the 10th of the month and financeCharge > 0 (i.e. the bill has not been paid on time).
<pre>checkBillStatus(LocalDate currentDate)</pre>	Same logic using the supplied date. Used for testing and presentation scenarios.
<pre>applyInterest()</pre>	Applies monthly APR to the current balance: currentPaymentDue += currentPaymentDue * ((interest_rate / 100) / 12).
<pre>makePayment(double amount) : double [override]</pre>	Logs a Transaction entry. Updates averageDue with the time-weighted balance since the last change. Sets dateSinceLastBalanceChange to today. Clears financeCharge first; any surplus is applied to principal via super.makePayment().
<pre>makePayment(double amount, LocalDate currentDate) : double</pre>	Same as above but uses the supplied date instead of today. Used for testing.
<pre>makeTransaction(double amount, String TransactionDesc)</pre>	Calls authorizeCharge(). If approved: updates averageDue, sets dateSinceLastBalanceChange to today, adds amount to currentPaymentDue, and records a Transaction. Prints the new balance.
<pre>makeTransaction(double amount, String TransactionDesc, LocalDate transactionDate)</pre>	Same as above using the supplied date. Used for testing and presentation.
<pre>getFinanceCharge() : double</pre>	Returns the current financeCharge value.

3.8 Customer - package User_Classes

File: src/User_Classes/Customer.java

Implements Comparable; sorted by lastName in an ArrayListManager. Note: the class has been renamed from the lowercase 'customer' in the original to 'Customer' (PascalCase), consistent with Java conventions.

Fields

Field	Type / Description
customerId	String - unique customer identifier (used as SSN/ID in CSV).
address	String.
city	String - added in v1.1 (CSV format is now 7 columns).
state	String - 2-letter code.
zipcode	String - 5-digit code.
firstName	String.
lastName	String - used as sort key.
accountList	List<Account> - accounts owned by this customer.

Methods

Method / Constructor	Description
<code>Customer()</code>	Default constructor - randomises customerId and zipcode; sets placeholder strings for other fields.
<code>Customer(String customerId, String address, String city, String state, String zipcode, String firstName, String lastName)</code>	Full parameterised constructor.
<code>LineForCSV() : String</code>	Returns a comma-delimited row of all 7 fields plus a newline.
<code>compareTo(Object o) : int</code>	Compares by lastName for sorted insertion.

3.9 AppState - package Utils

File: src/Utils/AppState.java

A final utility class that holds minimal shared in-memory state for the JavaFX UI. Declared final with a private constructor to prevent instantiation - it cannot be subclassed or instantiated. All fields are static, making them accessible across all controllers without passing object references between scenes. Intentionally kept lightweight while the team decides on a real persistence or service layer.

Fields

Field	Type / Description
customers (public static)	ArrayListManager<Customer> - the sorted list of all customers loaded from data.csv at startup. Accessed by all controllers via AppState.customers. Initialised as an empty ArrayListManager; populated by CsvManager.fetchCustsAndAccountsFromCSV().

3.10 ArrayListManager<T> - package Utils

File: src/Utils/ArrayListManager.java

A generic sorted collection utility. T must implement Comparable. Used as the primary data structure for the customer list (ArrayListManager<Customer>). Wraps an ArrayList<T> with sorted insertion, front/end insertion, removal by index, and a bubble-sort fallback. The mcount field tracks the number of elements independently of the underlying ArrayList size.

Fields

Field	Type / Description
mylist (protected)	ArrayList<T> - the underlying list.
mcount (protected)	int - count of elements currently stored.

Methods

Method	Description
<code>addAtEnd(T x) : int</code>	Appends x to the end of the list. Returns the new mcount.
<code>addInOrder(T x) : int</code>	Inserts x in sorted order using compareTo(). Handles head, tail, and mid-list insertion. Returns the new mcount.
<code>addAtFront(T x) : int</code>	Inserts x at index 0. Returns the new mcount.
<code>getValue(int i) : T</code>	Returns the element at index i. If i >= mcount, returns the first element as a safe fallback.
<code>getMcount() : int</code>	Returns the current element count.
<code>ManageAndSort()</code>	Bubble-sort pass over the list using compareTo(). Useful if elements are modified after insertion and the sort order has drifted.
<code>removeM(int i)</code>	Removes the element at index i and decrements mcount. Does nothing if i is out of bounds.

3.11 CheckpointManager - package Utils

File: src/Utils/CheckpointManager.java

Handles checkpointing the application state to disk. Called when the application needs to persist its current state - typically on exit or on a manual save trigger from the UI. Delegates all CSV I/O to CsvManager.

Methods

Method	Description
<code>EXEC_CHECKPOINT(ArrayListManager<Customer> CustomerList)</code>	Writes the full customer and account list to data.csv via CsvManager.writeCustomersToCsv(), then re-links all TMB overdraft accounts via CsvManager.handleOverdrafts(). Call this whenever the application exits or the user triggers a manual checkpoint.

<code>removeCustFromCsv(Customer Customer)</code>	Stub - intended to remove a single customer record from the CSV. Not yet implemented.
---	---

Note: CheckpointManager has been promoted from 'Beyond Prototype Scope' to implemented in this version. It is no longer a planned class.

3.12 CsvManager - package Utils

File: src/Utils/CsvManager.java

Static utility class that owns all CSV read and write operations. Handles both customer and account data in a single combined CSV file (src/data.csv). Account records are stored inline on the same row as the customer, pipe-delimited (|) after the 7 customer columns. CsvManager is used by CheckpointManager and the JavaFX controllers.

Methods

Method	Description
<code>fetchCustsAndAccountsFromCSV(ArrayListManager<Customer> CustomerList)</code>	Overload - calls <code>fetchCustsAndAccountsFromCSV(CustomerList, "src/data.csv")</code> using the default path.
<code>fetchCustsAndAccountsFromCSV(ArrayListManager<Customer> CustomerList, String path)</code>	Reads the CSV at the given path. Each row is parsed into a Customer (first 7 columns) and zero or more Account objects (columns 8+ split by comma, fields within each account split by pipe). Account type is determined by the first pipe-field and the correct subclass is instantiated via switch. Accounts are added to the customer's accountList. The customer is inserted into CustomerList in sorted order.
<code>handleOverdrafts(ArrayListManager<Customer> CustomerList)</code>	After loading, iterates all customers and all accounts. For each TMBAccount, uses <code>overdraftIdentifier</code> (an index into accountList) to locate the corresponding SavingsAccount and calls <code>setOverdraftProtAccount()</code> to re-link the overdraft relationship. Restores overdraft links that were saved to CSV.
<code>removeCustFromArrBySSN(ArrayListManager<Customer> CustomerList, String SSN)</code>	Searches CustomerList by customerId and removes the matching entry. Prints a confirmation or 'cust not found' if no match.
<code>writeCustomersToCsv(ArrayListManager<Customer> CustomerList)</code>	Writes the entire CustomerList back to src/data.csv. Each customer row includes all 7 customer fields followed by pipe-delimited account records for each account in accountList. Supports SavingsAccount, TMBAccount, GDAccount, and CDAccount. FileWriter overwrites the existing file.

CSV Format - Combined Customer + Account Row

Each row in data.csv follows this structure:

```
customerId,address,city,state,zip,firstName,lastName,AccountType|field1|field2|...,AccountType|field1|...
```

Account Type	Pipe-delimited fields after type name
--------------	---------------------------------------

SavingsAccount	interestRate compoundFreq overdraftBackup (1/0) balance
TMBAccount	balance overdraftIdentifier (index into accountList, -1 if none)
GDAccount	balance dailyRateFlexible (true/false)
CDAccount	balance fixedRate maturityDate earlyPenalty

3.13 PrintUtil - package Utils

File: src/Utils/PrintUtil.java

Static utility for writing a single customer and account record to customers.csv. Used in the developer testing area (Main.java) to demonstrate file output. Not part of the production checkpoint flow - use CsvManager and CheckpointManager for that.

Methods

Method	Description
<code>saveCustomerData(Customer customer, Account account, String extraData)</code>	Writes a single CSV row to customers.csv containing: customerId, firstName, lastName, accountNumber, balance, extraData. Overwrites the file on each call. Prints a confirmation or error message to stdout.

3.14 Transaction - package Utils

File: src/Utils/Transaction.java

A simple value object representing a single financial event on a CreditCard account. Created by CreditCard.makePayment() and CreditCard.makeTransaction() and stored in the CreditCard.transactions list. Provides an audit trail of all charges and payments.

Fields

Field	Type / Description
amount	double - dollar amount of the transaction.
transactionId	int - sequential ID, set to transactions.size() at time of creation.
TransactionDate	LocalDate - date the transaction was recorded. Set to LocalDate.now() at construction.
description	String - label for the transaction (e.g. "Balance Payment", merchant name).

Methods

Method / Constructor	Description
<code>Transaction(double amount, int transactionId, String description)</code>	Sets all fields. TransactionDate is always set to today.

```
toString() : String
```

Returns transactionId + "," + description + "," + amount - used for logging and display.

3.15 Main - default package

File: src/Main.java

Developer testing area only. Not part of the production application flow. Used during development to manually exercise CSV I/O and account behaviour. The actual application entry point is MainApp.java.

3.16 MainApp - default package

File: src/MainApp.java

JavaFX entry point for the production application. Launches MyApp (an inner Application subclass) which loads SystemController.fxml, creates a Scene, and opens the primary Stage titled "System Controller". This is the class that should be run to start the application.

3.17 Beyond Prototype Scope

The following classes are fully specified in the UML diagrams but are not yet implemented in prototypeBranch. They remain documented in the UML for design completeness.

Class / Component	Intended Role (from UML)
RecoveryManager	Disaster recovery orchestration; getLastGoodBalance() for fee waiver.
CheckpointStorage	Durable checkpoint record store with integrity verification.
BankingSystem	Central facade routing all transactions.
Teller / BankManager	Role-based actor classes with escalating operation privileges.
ATMCard	Daily withdrawal limit enforcement linked to a customer.

4. Data Layer

The prototype uses a single flat CSV file (src/data.csv) for all persistence. Customer and account data are stored together - each row contains a customer record followed by zero or more account records on the same line. All CSV I/O is handled by CsvManager. CheckpointManager calls CsvManager to save state. AppState holds the in-memory customer list shared across the UI.

4.1 data.csv - Customer Seed Data

Location: src/data.csv | 15 pre-populated records

Column	Field
0	customerId - numeric SSN/ID string
1	Address - street address
2	City - city name
3	State - 2-letter code
4	Zip - 5-digit ZIP code
5	First - first name
6	Last - last name

4.2 Account Data - Combined Row Format

Account data is stored inline on the same CSV row as the owning customer, separated by commas. Each account is encoded as a pipe-delimited (|) token starting with the account type name. CsvManager.fetchCustsAndAccountsFromCSV() parses columns 8+ as account tokens and uses a switch on the type name to instantiate the correct subclass. The full format per row is:

```
customerId,address,city,state,zip,firstName,lastName,AccountType|f1|f2|...,AccountType  
|f1|f2|...
```

Account Type	Pipe-delimited fields (after type name)
SavingsAccount	interestRate compoundFreq overdraftBackup (1=true, 0=false) balance
TMBAccount	balance overdraftIdentifier (index into accountList, -1 if no overdraft account)
GDAccount	balance dailyRateFlexible (true/false)
CDAccount	balance fixedRate maturityDate earlyPenalty

After loading, CsvManager.handleOverdrafts() iterates all TMBAccounts and uses overdraftIdentifier as an index into the customer's accountList to re-link the overdraft SavingsAccount relationship that was saved to disk.

4.3 CSV Parsing - CsvManager

CsvManager.fetchCustsAndAccountsFromCSV() is the authoritative loader. It reads src/data.csv line by line via a try-with-resources Scanner, splits on commas (limit -1 to preserve trailing empty fields), and calls CustomerList.addInOrder() with a fully parameterised Customer. Account tokens from column 8 onward are split by pipe and dispatched via switch to the correct constructor.

```
try (Scanner fileReader = new Scanner(new File("src/data.csv"))) {
    while (fileReader.hasNextLine()) {
        String[] fields = fileReader.nextLine().split(",");
        // fields: [0]=id [1]=addr [2]=city [3]=state [4]=zip [5]=first [6]=last
        CustomerList.addInOrder(new Customer(fields[0], fields[1], fields[2],
                                              fields[3], fields[4], fields[5],
fields[6]));
    }
}
```

4.4 Hypothetical: Production Data Layer

If this prototype were extended into a production system, the CSV layer would be replaced with a relational database (e.g., PostgreSQL) with separate tables for customers, accounts, transactions, and loans, accessed through a repository pattern. CheckpointStorage would persist serialised account snapshots as byte arrays with SHA-256 checksums for integrity verification.

5. Project Structure & Build

5.1 Repository Layout

```
Banking-System/
├── src/
│   ├── Account_Classes/
│   │   ├── Account.java
│   │   ├── SavingsAccount.java
│   │   ├── CDAccount.java
│   │   ├── CheckingsAccount.java
│   │   ├── TMBAccount.java
│   │   └── GDAccount.java
│   ├── Loan_Classes/
│   │   ├── Loan.java
│   │   ├── TermBasedLoan.java
│   │   ├── MortgageLoan.java
│   │   ├── ShortTermLoan.java
│   │   └── CreditCard.java
│   ├── User_Classes/
│   │   └── Customer.java
│   ├── Utils/
│   │   ├── AppState.java           # Global shared state for JavaFX controllers
│   │   ├── ArrayListManager.java  # Generic sorted collection
│   │   ├── CheckpointManager.java # State persistence on exit / manual save
│   │   ├── CsvManager.java        # All CSV read/write operations
│   │   ├── PrintUtil.java         # Dev-only single-record file writer
│   │   └── Transaction.java       # Value object for CreditCard ledger
│   ├── Main.java                  # Developer testing area only
│   ├── MainApp.java               # JavaFX production entry point
│   └── data.csv                   # Combined customer + account seed data
├── tests/
│   ├── AccountTests.java
│   ├── LoanTests.java
│   └── MainTests.java
├── lib/
│   └── junit-platform-console-standalone-1.13.0-M3.jar
└── Banking-System.iml
```

5.2 Building & Running

IntelliJ IDEA

- Open the project root. Confirm JDK 11+ under File → Project Structure → SDK.
- Verify the JUnit JAR is listed under Project Libraries.
- Build: Ctrl+F9 (Cmd+F9 on Mac).
- Run Main: right-click src/Main.java → Run 'Main.main()'.
- Run GUI: right-click src/MainApp.java → Run 'MainApp.main()' (requires JavaFX SDK on module path).

Command Line (Main - CSV demo)

```
javac -d out/production/Banking-System src/Account_Classes/*.java \  
      src/Loan_Classes/*.java src/User_Classes/*.java src/Utils/*.java src/Main.java  
java -cp out/production/Banking-System Main
```

6. Contributing Guidelines

6.1 Branching Strategy

Branch	Purpose
main	Stable prototype. All changes enter via pull request.
prototypeBranch	Current active development branch. Source of truth for this manual version.
feature/[name]	New class or feature (e.g., feature/checkpoint-manager). Branch from main.
fix/[name]	Bug fixes against the current prototype.
test/[name]	New or expanded test coverage only.

6.2 Pull Request Standards

- At least one other team member (Team Lead) must review before merging.
- New classes should include at least one JUnit test in the appropriate test file.
- The project must compile cleanly before opening a PR.
- New classes belong in their own .java file in the appropriate package under src/ - not in Main.java.
- Any deviation from the UML class diagram must be noted in the PR description and communicated to the Documentation lead (Aidan Godair) for manual update.

6.3 UML Alignment

Implemented classes should align with the UML Class Diagram in Section 2.1. Where the prototype intentionally deviates (e.g., requestFeeWaiver() using a local balance check rather than CheckpointManager), the deviation is noted in Section 3 of this manual.

7. Hypothetical: API Design

No REST API is implemented in the current codebase. The design below describes how an API layer would be structured if BankApp were extended beyond the prototype.

7.1 Design Principles

- All endpoints follow REST conventions: nouns for resources, HTTP verbs for actions.
- Responses use a standard JSON envelope: { "status", "data", "error" }.
- API versioning via URL prefix: /api/v1/...

7.2 Representative Endpoints

Method + Path	Description
POST /api/v1/accounts	Create a new account for a customer.
GET /api/v1/accounts/{id}	Retrieve account details and current balance.
POST /api/v1/accounts/{id}/deposit	Deposit an amount into the specified account.
POST /api/v1/accounts/{id}/withdraw	Withdraw an amount from the specified account.
GET /api/v1/customers/{id}	Retrieve customer profile.
POST /api/v1/transactions	Initiate a transfer between two accounts.
GET /api/v1/accounts/{id}/statement	Return transaction history for an account.
POST /api/v1/loans/{id}/payment	Make a payment against a loan or credit card.

8. Hypothetical: Security

Security is out of scope for the prototype. The following describes the security model that would be applied in a production extension of BankApp, consistent with the role-based actor design shown in the UML Use Case Diagram.

8.1 Authentication

- JWT (JSON Web Tokens) would be issued on successful login and included in the Authorization header.
- OAuth 2.0 could be layered on for third-party integrations.

8.2 Role-Based Access Control

Role	Permitted Operations
Customer	Own account deposits, withdrawals, balance queries, CC purchases, loan payments.
Teller	All Customer operations plus account credit/debit, stop payments, fund transfers.
Bank Manager	All Teller operations plus interest rate management, CD rollovers, loan oversight.
System	Checkpoint save/restore, health monitoring, scheduled billing.

8.3 Data Protection

- All data in transit encrypted via TLS 1.3.
- Sensitive fields (SSN, account numbers) encrypted at rest with AES-256.
- The prototype's data.csv stores SSNs in plain text - acceptable for academic demonstration only.

9. Hypothetical: CI/CD & Deployment

No CI/CD pipeline exists for the prototype. This section outlines what would be implemented for a deployed service.

Stage	Action
Trigger	Push to any branch or pull request opened against main.
Build	javac compilation; fail pipeline on any compile error.
Test	Run AccountTests, LoanTests, and MainTests via JUnit. Fail on any test failure.
Package	Produce a runnable JAR using Maven or Gradle.
Deploy	Push JAR to staging; promote to production on manual approval (main branch only).

10. Error Handling & Logging

10.1 Current Prototype Approach

- FileNotFoundException is caught in Main.main() when data.csv cannot be found.
- All balance changes, interest applications, and overdraft events are printed via System.out.println().
- No logging framework is used - console output is sufficient for academic demonstration.

10.2 Hypothetical: Production Error Handling

- SLF4J with a Logback backend would replace System.out.println() calls.
- A global exception handler (e.g., @ControllerAdvice in Spring) would return structured error responses.
- Typed exceptions (e.g., InsufficientFundsException, AccountNotFoundException) would replace null returns.
- Audit logging would record every transaction with timestamp, actor, account ID, and outcome.

11. Testing

The prototype now includes three JUnit 5 test files covering accounts, loans, and file I/O.

11.1 AccountTests.java

File: tests/AccountTests.java

Sets up a TMBAccount (balance \$1,000) and a SavingsAccount (rate 25%, balance \$2,000) linked to a default Customer before each test via `@BeforeEach`.

Test	Verifies
<code>testDeposit()</code>	Depositing \$500 into the checking account raises balance to \$1,500.
<code>testWithdraw()</code>	Withdrawing \$1,000 leaves a balance of \$0.
<code>testSetOverdraftProtection()</code>	<code>setOverdraftProtAccount()</code> executes without throwing.
<code>testAccountFundTransfer()</code>	Transferring \$500 from checking to savings raises savings balance to \$2,500.
<code>testOverdraftPenalty()</code>	Withdrawing \$1,100 (overdraft) then calling <code>handleOverdraft()</code> with no backup account charges a \$25 fee, leaving balance at $-\$125$.
<code>testOverdraftFunctionality()</code>	After overdraft, setting the savings account as backup and calling <code>handleOverdraft()</code> transfers the deficit, leaving savings at \$1,900.
<code>testSavingsInterest()</code>	<code>applyDailyInterest()</code> on a \$2,000 account at 25% rate raises balance to \$2,500.

11.2 LoanTests.java

File: tests/LoanTests.java

Exercises MortgageLoan, ShortTermLoan, and CreditCard with specific date parameters to enable deterministic assertions.

Test	Verifies
<code>testLoanCreation()</code>	MortgageLoan (15yr) and ShortTermLoan instantiate without throwing.
<code>testLoanPayment()</code>	After one payment cycle and a \$50 payment on a 15yr/5% \$25k mortgage, balance truncates to \$147.
<code>testLoanRepayment()</code>	After one cycle on a 30yr/15% \$10k mortgage, balance rounds to \$127.
<code>testFullExampleLoan()</code>	180 full payment cycles on a 15yr/6% \$30k mortgage closes the account at \$0.
<code>testOverpaidExampleLoan()</code>	Same loan with an extra \$500 payment at cycle 40 still closes at \$0, verifying schedule recalculation.

testLoanProblemFlagMissed()	A missed payment correctly sets isProblemAccount = true when flagIfMissed() is called with a future date.
testLoanProblemFlagPaid()	A paid account is not flagged as a problem account.
testCreditCardCharge()	authorizeCharge(\$100) returns true when under the \$300 limit.
testCreditCardOvercharge()	authorizeCharge(\$400) returns false when over the \$300 limit.
testApplyInterest()	applyInterest() on a \$300 balance with 0% rate leaves balance at \$300.
testGenerateBill()	Finance charge after a partial payment cycle equals \$200.
testCheckBillStatus()	An unpaid bill on or after the 10th sets isProblemAccount = true.

Note: testCreditCardCreation() is currently commented out because Customer.accountList cannot hold Loan objects without a cast - a known limitation to address in a future iteration.

11.3 MainTests.java

File: tests/MainTests.java

Retains the original smoke test:

Test	Verifies
testFileExists()	src/data.csv is present and can be opened with Scanner without throwing.

11.4 Running Tests

IntelliJ IDEA

Right-click any test file → Run. All three files can be run together via the test source root configuration.

Command Line

```
javac -cp lib/junit-platform-console-standalone-1.13.0-M3.jar \
      -d out/production/Banking-System tests/*.java src/**/*.java src/*.java
java -cp
out/production/Banking-System:lib/junit-platform-console-standalone-1.13.0-M3.jar \
    org.junit.platform.console.ConsoleLauncher --select-package=tests
```

12. Controllers

BankApp's JavaFX UI is built around a set of controller classes located in **UI/Controllers/**. Each controller backs one FXML scene and handles all user interactions for that screen. Controllers access shared application state through **AppState** and persist changes via **CsvManager**. The table below documents every controller on prototypeBranch.

12.1 Controller Summary

Controller	Description
ATMWithdrawController	Handles ATM cash withdrawals for customers. Lets the user select a customer and account, validates the requested amount against available funds (including CD early-withdrawal penalties), executes the withdrawal, triggers overdraft handling for checking accounts, and persists the updated balances to CSV.
CheckPointController	Provides checkpoint and recovery controls for system administrators. Supports saving the current in-memory state to CSV, restoring a previously saved state from disk, and performing a final checkpoint-then-exit shutdown sequence.
CreateNewAccountController	Allows a teller to open a new bank account (Savings, Gold Diamond Checking, TMB Checking, or CD) for an existing customer, collecting the appropriate parameters for each account type and writing the new account to CSV.
CreateNewCreditCardController	Allows a teller to issue a new credit card to an existing customer by specifying a credit limit, interest rate, and optional starting balance, then attaches the card to the customer's payoff list and saves the record.
CreateNewCustomerController	Registers a new customer by collecting all required fields (customer ID, name, address, city, state, ZIP), validates for duplicates and format, inserts the customer into the in-memory sorted list, and persists to CSV.
CreateNewLoanController	Allows a bank manager to create a new Mortgage Loan or Short Term Loan for an existing customer, accepting principal, interest rate, grace period, and (for mortgages) term, then attaches the loan to the customer and saves.
CreditCardScreenController	Supports customer-facing credit card operations: searching for a customer's cards by ID, making payments from a linked bank account, and charging purchases to the card, updating balances and persisting transactions after each action.
CustomerScreenController	Navigation hub for the Customer role. Provides buttons to navigate to the Credit Card screen, Review Account Status screen, Input Check screen, and ATM Withdraw screen, as well as a return to the System Controller.
DepositToAccountController	Allows a teller to deposit a specified amount into any of a customer's accounts. Displays previous balance, deposit amount, and new balance after the transaction, then saves updated data to CSV.

Controller	Description
EstablishCDsController	Allows a bank manager to open a new Certificate of Deposit for a customer by selecting a term (6, 12, 24, or 36 months), initial deposit, interest rate, and early-withdrawal penalty, then calculates the maturity date and saves the CD.
InputCheckScreenController	Placeholder screen for the customer check-input workflow. Currently contains only navigation logic to return to the Customer Screen; no transactional functionality is implemented.
LinkAccountsController	Allows a teller to link a customer's checking account (TMB or Gold Diamond) to one of their savings accounts as an overdraft backup, calling <code>setOverdraftProtAccount()</code> on the checking account and persisting the relationship.
LoadBasicDataController	Provides system-level data loading and validation utilities. Can load customer and account data from a specified CSV path into <code>AppState</code> , or scan the file row-by-row to report any improperly formatted records.
ManagerScreenController	Navigation hub for the Bank Manager role. Provides buttons to navigate to Create New Loan, Set Loan Interest Rates, Set CD and Checking Rates, Establish CDs, and Process Checks screens.
ProcessChecksController	Placeholder screen for the check-processing workflow accessible to the Bank Manager. Currently contains only navigation logic to return to the Manager Screen; no transactional functionality is implemented.
ProgressTimeController	System-level utility that advances the application's internal timeline by a user-specified number of days, updating <code>AppState.timeline</code> so that interest calculations, loan cycles, and billing events reflect the new simulated date.
ReviewAccountStatusController	Customer-facing account review screen. Requires the customer to authenticate with their ID, first name, and last name, then displays a detailed breakdown of all their savings, checking, CD, credit card, and loan accounts, plus ATM withdrawal count.
ReviewCustomerAccountsController	Teller-facing account review screen. Allows staff to select any customer from a dropdown and view a full summary of all accounts and loans, including balances, rates, linked accounts, transaction history, and problem-account flags.
RunSimpleRegressionTestsController	Placeholder screen for running regression tests from within the UI. Currently contains only navigation logic to return to the System Controller; no test-execution functionality is implemented.
ScreenNavigator	Shared static utility class (not a JavaFX controller) that provides a single <code>switchScreen()</code> method used by multiple controllers to load an FXML file, set the Stage scene, and update the window title in one call.
SetCDAndCheckingRatesController	Allows a bank manager to set the system-wide default interest rate for new Savings Accounts, CD Accounts, and Gold Diamond Checking Accounts, storing the rate and its effective date in <code>AppState</code> for use when those accounts are created.

Controller	Description
SetLoanInterestRatesController	Allows a bank manager to set the system-wide default interest rate for new Mortgage Loans and Short Term Loans, storing the rate and its effective date in AppState so new loans use the current manager-approved rate.
StopPaymentController	Placeholder screen for the stop-payment workflow accessible to the Teller. Currently contains only navigation logic to return to the Teller Screen; no stop-payment functionality is implemented.
SystemControllerController	Root navigation hub for the application. Provides access to all three role screens (Teller, Manager, Customer) as well as system-level screens for checkpoint management, data loading, regression testing, and time progression.
TellerScreenController	Navigation hub for the Teller role. Provides buttons to navigate to Create New Account, Create New Customer, Link Accounts, Withdraw from Account, Deposit to Account, Stop Payment, Review Customer Accounts, and Create New Credit Card screens.
WithdrawFromAccountController	Allows a teller to process a cash withdrawal from any of a customer's accounts. Validates sufficient funds (including CD early-withdrawal penalty), executes the withdrawal, invokes overdraft handling for checking accounts, and saves results to CSV.

13. Appendices

Appendix A - Team

Name	Role
Macalister Clark	Team Lead
Aidan Godair	Documentation
Carson Souther	Programmer
Logan Sullivan	Programmer
Michael Bowen	Programmer
Ike Hayes	Test Developer
Matthew Fuller	Test Developer

Appendix B - Dependencies

Dependency	Version / Purpose
Java	JDK 11 or higher - language and runtime.
IntelliJ IDEA	IDE used for development (community or ultimate edition).
JavaFX SDK	Required for MainApp GUI launcher (version must match JDK).
JUnit Platform Console Standalone	1.13.0-M3 - test runner for all three test files.

Appendix C - data.csv Full Record List

The following 15 customer records are included in src/data.csv. Note the updated 7-column format (adds City vs. v1.0's 6-column format).

423453245,114 North 4th,St. Joseph,MO,64493,Ronald,Jones
345653425,1805 Jules,St. Joseph,MO,64503,Mark,Ingrem
423148894,2109 Sylviana,St. Joseph,MO,64501,Sherry,Harrison
677879013,4305 West 3rd,Kansas City,KS,54910,John,Morgeson
345599870,221 South Highly,St. Joseph,MO,64503,Gary,Gilkerson
423567890,335 Jules,St. Joseph,MO,64503,Mary,Francis
345812332,1402 Mocking Bird,St. Joseph,MO,64502,Carolyn,Johnson
345879912,2105 Senior Dr.,St. Joseph,MO,64503,Larry,Waite

423168945, 1801 Lovers Ln, St. Joseph, MO, 64502, Marilyn, Macklin
 235445789, 2508 Stonecrest, St. Joseph, MO, 64501, Lana, McGlynn
 436789213, 4156 N. Mulbury, St. Joseph, MO, 64502, John, Munzer
 233435656, 15525 Kioa, Kansas City, KS, 56708, Mary, White
 435678787, 8902 Martha Ln, St. Louis, MO, 63301, Jerry, Gronius
 563432913, 1532 Lovers Ln, St. Joseph, MO, 64502, Martin, Williams
 345667989, 8703 Park Ln, St. Joseph, MO, 64501, Broderick, Jones

Appendix D - UML Diagram Index

Figure	Diagram
Figure 1	UML Sequence Diagram - Scenarios A-H (Section 2.3)
Figure 2	UML Activity Diagram - Transaction Flow & Recovery (Section 2.4)
Figure 3	UML Use Case Diagram - Banking System (Section 2.2)
Figure 4	UML Class Diagram - Full System Hierarchy (Section 2.1)

Appendix E - Key Changes from v1.1 to v1.2

This appendix summarises the most significant differences between v1.1 and v1.2, reflecting the project lead's feedback, the latest prototypeBranch code, and the newly documented Utils package.

Area	Change
Loan section formatting (3.7)	Loan, TermBasedLoan, and CreditCard sections now each have separate 'Fields' and 'Methods' subheadings with individual tables, matching the formatting style of the Account sections.
CreditCard methods (3.7d)	All CreditCard methods are now fully explained with detailed descriptions covering behaviour, parameters, return values, and when to call each method. Previously listed without explanation.
Main.java description (2.1, 3.10)	Main is now correctly documented as a developer testing area only, not the production entry point. All listed method documentation removed as it is not part of the application API.
AppState class (2.1, 3.9)	New class added and documented. AppState is a singleton in the Utils package that holds global data structures (primarily ArrayListManager) shared across all JavaFX controllers.
Section numbering (3.9-3.12)	Main moved to 3.10, MainApp to 3.11, Beyond Prototype Scope to 3.12 to accommodate the new AppState section at 3.9.

Area	Change
Account CSV parsing (4.2)	New section added documenting the account CSV format and parsing logic. Account records are now read from CSV at startup and linked to their owning Customer via ownerId. Section numbering in Data Layer updated accordingly.
UML diagrams	All four diagrams (Class, Use Case, Sequence, Activity) replaced with the original high-quality source images uploaded by the team.
Sections removed	Section 11.5 (Recommended Tests for Future Extensions) and the Activities / Next Week / Problems section removed from this version of the manual.
Utils package (new, 3.9-3.14)	Six Utils classes now fully documented: AppState (global state singleton), ArrayListManager (generic sorted collection), CheckpointManager (promoted from Beyond Scope - now implemented), CsvManager (new - owns all CSV I/O replacing scattered Main methods), PrintUtil (dev-only file writer), and Transaction (CreditCard value object).
CsvManager (new, 3.12)	Major new class. Handles combined customer+account CSV format where accounts are stored inline pipe-delimited on the customer row. Replaces the CSV methods previously scattered in Main.
CheckpointManager (3.11)	Promoted from 'Beyond Prototype Scope' to fully implemented. EXEC_CHECKPOINT() calls CsvManager to persist state. removeCustFromCsv() is a stub.
Data layer (Section 4)	Updated to document the combined customer+account CSV row format and pipe-delimited account encoding. Section 4.2 now describes the real CsvManager parsing logic, replacing the placeholder account CSV format from v1.2.

Appendix F - Key Changes from v1.2 to v1.3

This appendix summarises the most significant differences between v1.2 and v1.3, reflecting the addition of the Controllers section and resulting structural updates.

Area	Change
Section reordering (12 & 13)	Controllers section moved to Section 12 and Appendices moved to Section 13, placing implementation documentation before reference appendices for a more logical reading order.
Controllers section (12)	New Section 12 added documenting all 26 JavaFX controller classes in UI/Controllers/ on prototypeBranch. Each controller entry describes its role, key interactions, and whether it is fully implemented or a navigation/placeholder stub.
Appendix E updated	Key Changes appendix now includes a v1.2 -> v1.3 entry summarising the Controllers section addition and section renumbering.
Version bump	Manual version updated from v1.2 to v1.3 on the cover page and status line.